

```
[1]: a = [1,2,3,4,5]
      print(a)
```

```
[1, 2, 3, 4, 5]
```

```
[3]: import numpy as np
      arr=np.array([1,2,3,4,5])
      print(arr)
```

```
[1 2 3 4 5]
```

```
[4]: a=[1,2,3]
      b=[4,5,6]
      res=[]
      for i in range(len(a)):
          res.append(a[i]+b[i])
      print(res)
```

```
[5, 7, 9]
```

```
[6]: import numpy as np
      a=np.array([1,2,3])
      b=np.array([4,5,6])
      print(a+b)
      print(a-b)
      print(a*b)
      print(a**b)
```

```
[5 7 9]
```

```
[-3 -3 -3]
```

```
[ 4 10 18]
```

```
[ 1 32 729]
```

```
[44]: import numpy as np
arr=np.array([1,2,3,4,5])
ar=np.array([[1,2,3],[4,5,6]])
a=np.array([[[1,2],[3,4],[4,5]]])
print(arr)
print(ar)
print(a)
print(arr.shape) #to identify rows and columns
print(ar.shape)
print(a.shape)
print(arr.ndim) #to identify no of rows alone
print(ar.ndim)
print(a.ndim)
print(arr.size) #to identify total no of elements
print(ar.size)
print(a.size)
print(arr.dtype) #it shows data type
print(ar.dtype)
print(a.dtype)
```

```
[1 2 3 4 5]
```

```
[[1 2 3]
```

```
 [4 5 6]]
```

```
[[[1 2]
```

```
 [3 4]
```

```
 [4 5]]]
```

```
(5,)
```

```
(2, 3)
```

```
(1, 3, 2)
```

```
1
```

```
2
```

```
3
```

```
5
```

```
6
```

```
6
```

```
int32
```

Array creation in numpy

```
[39]: import numpy as np
a=np.array([1,2,3,4,5])
b=np.zeros(5)
print(a)
print(b)
c=np.zeros((2,2)) #change to 0's
print(c)
d=np.ones((3)) #shape of 1d array
print(d)
e=np.ones((2,3)) #shape of 2d array that convert to one's
print(e)
f=np.ones((1, 3, 2))
print(f)
print(np.arange(1,10,2)) #works like range
print(np.linspace(0,20,5)) #decimal values print
print(np.linspace(0,20,6))
print(np.random.rand(5)) #it shows only 5 random numbers
print(np.random.randint(1,100,7)) #it shows integer number only 7 numbers from 1 to 100
print(np.random.randn(5)) #print both positive and negative
```

```
[1 2 3 4 5]
[0. 0. 0. 0. 0.]
[[0. 0.]
 [0. 0.]]
[1. 1. 1.]
[[1. 1. 1.]
 [1. 1. 1.]]
[[[1. 1.]
   [1. 1.]]
 [1. 1.]]]
[1 3 5 7 9]
```

```

[ 0.  5. 10. 15. 20.]
[ 0.  4.  8. 12. 16. 20.]
[0.61243114 0.68631482 0.12161291 0.25210412 0.66843295]
[67 98 46 40 72 55 30]
[-1.29083681 -0.51043769  0.04044688 -0.06021058 -0.26716552]

```

Numpy indexing and slicing

```

[66]: import numpy as np
a=np.array([10,20,30,43,50,60,67])
#1d array indexing
print(a[2])
print(a[-1])
b=np.array([[1,2,3],
            [4,5,6],
            [7,8,9]])
#2d array indexing
print(b[1][1])
#1d slicing
print(a[2:7])
print(a[:4])
print(a[::2])
print(a[::1])
#2d slicing
print(b[1]) #first row alone
print(b[0])
print(b[:,1]) #column slice    ',' is used for column slicing    '1' is used to get first column value
print(b[:,-1]) #negative index and last it print
#boolean
print(a[a>50]) #boolean filter based on condition
print(a[a<50])
#fancy indexing
print(a[[0,4,3,6]]) #these indexing value should be printed here
print(a[a%2==0]) #this is boolean indexina and print onlv even numbers

```



```
a[4]=1000
print(a)
```

```
30
67
5
[30 43 50 60 67]
[10 20 30 43]
[10 30 50 67]
[10 20 30 43 50 60 67]
[4 5 6]
[1 2 3]
[2 5 8]
[3 6 9]
[60 67]
[10 20 30 43]
[10 50 43 67]
[10 20 30 50 60]
[ 10  20  30  43 1000  60  67]
```

mathematical operation in numpy

```
[70]: import numpy as np
a=np.array([1,2,3])
print(a+10) #add 10 to all the array values
b=np.array([[1,2,3],
            [4,5,6]])
print(b+10) #2d broadcasting
print(b+a)  #row broadcasting
#always no of columns must be same
```

```
[11 12 13]
[[11 12 13]
 [14 15 16]]
[[2 4 6]
 [5 7 9]]
```

matrix

```
[75]: import numpy as np
a=np.array([[1,2],
            [3,4]])
b=np.array([[4,3],
            [2,1]])
print(a*b)
print(np.dot(a,b))
print(a@b) #alternative for dot operator
print(np.matmul(a,b)) #alternative method of dot
print(a.T) #row become column and column became row
```

```
[[4 6]
 [6 4]]
[[ 8  5]
 [20 13]]
[[ 8  5]
 [20 13]]
[[ 8  5]
 [20 13]]
[[1 3]
 [2 4]]
```

universal mathematical function

```
[82]: import numpy as np
a=np.array([4,9,16])
print(np.sqrt(a))
b=np.array([-5,-6,-2,-8])
print(np.abs(b)) #to change negative to positive
print(np.log(a)) #used to find log value
print(np.exp(a))
```

```
[5 6 2 8]
[1.38629436 2.19722458 2.77258872]
[5.45981500e+01 8.10308393e+03 8.88611052e+06]
```

round function

```
[87]: arr=np.array([1.234,5.674])
print(np.round(arr,2)) #only two values it will be print
print(np.floor(arr))
print(np.ceil(arr))
```

```
[1.23 5.67]
[1. 5.]
[2. 6.]
```

statistical function

```
[107]: import numpy as np
a=np.array([10,2,3,40,50])
print(np.mean(a))
print(np.median(a)) #after sorting the middle value will return as median
print(np.std(a)) #standard deviation value will be printed
print(np.min(a)) #min value printed
print(np.max(a)) #max value will be printed
print(np.percentile(a,50)) #after sorting the value will print
print(np.percentile(a,25))
print(np.percentile(a,75))
print(np.var(a)) #to find variance
print(np.argmin(a)) #index min value
print(np.argmax(a)) #index max value
b=np.array([1,2,3,4,5])
c=np.array([10,8,6,4,2])
print(np.corrcoef(b,c)) #corelation checking
```

```

21.0
10.0
20.039960079800558
2
50
10.0
3.0
40.0
401.6
1
4
[[ 1. -1.]
 [-1.  1.]]

```

Aggerate function

```

[112]: d=np.array([[1,3,4],
                  [5,6,8]])
print(np.sum(d))
print(np.sum(d,axis=0))
print(np.sum(d,axis=1))

27
[ 6  9 12]
[ 8 19]

```

▼ question ¶

```

[125]: import numpy as np
scores=np.array([[95,97,80,67,56],
                [78,75,70,90,98]])
print(np.mean(scores))
print(np.max(scores,axis=1)) #column
normalized =(scores-np.min(scores))/(np.max(scores)-np.min(scores))

```


question

```
[125]: import numpy as np
scores=np.array([[95,97,80,67,56],
                 [78,75,70,90,98]])
print(np.mean(scores))
print(np.max(scores,axis=1)) #column
normalized =(scores-np.min(scores))/(np.max(scores)-np.min(scores))
print(normalized)
print(np.mean(scores,axis=0)) #row 95+78 like this
```

80.6
 [97 98]
 [[0.92857143 0.97619048 0.57142857 0.26190476 0.]
 [0.52380952 0.45238095 0.33333333 0.80952381 1.]]
 [86.5 86. 75. 78.5 77.]

```
[132]: import numpy as np
arr=np.array([10,20,30,40,50])
s=(arr[arr>25])
print(s)
total=sum(s)
print(total)
```

[30 40 50]
 120